Asistente RAG reproducible sobre prensa y Bluesky con Milvus y modelos locales: arquitectura, evaluación y compuerta de dominio

Erika Valeria Rivera Jimenez Pontificia Universidad Javeriana
Bogotá, Colombia
riveraev@javeriana.edu.co Julian Guillermo Carrillo Meneses Pontificia Universidad Javeriana
Bogotá, Colombia
carrillojuliang@javeriana.edu.co Carlos Andres Garnica Salazar Pontificia Universidad Javeriana
Bogotá, Colombia
carlos_garnicas@javeriana.edu.co Oscar Andres Prieto Moreno Pontificia Universidad Javeriana
Bogotá, Colombia
o-prieto@javeriana.edu.co

***Resumen*: La recuperación aumentada por generación (RAG) permite responder sobre un corpus sin reentrenar el modelo, pero su calidad depende de decisiones que rara vez se documentan juntas: limpieza, segmentación, codificador, índice, reordenamiento, prompt y control de abstención. Este trabajo presenta un asistente RAG reproducible sobre 169.248 documentos recolectados de prensa colombiana y Bluesky, 161.868 indexados tras descartar páginas de error y 618.286 fragmentos en Milvus, con modelos locales. Se justifica cada capa con experimentos pareados: la limpieza conservadora no mejora de forma significativa, el chunking fijo supera a la segmentación por oraciones, bge-m3 supera a e5-base y a la fusión con BM25, y un cross-encoder sobre el top-5 sube el nDCG de 0,809 a 0,865 y el P@1 de 0,737 a 0,868 sin perder cobertura. El sistema final entrega 37 de 38 preguntas respondibles, se abstiene en las 5 sin respuesta y acierta 33 de 37 en revisión manual. En conversación alcanza 14 de 14 con compuerta de dominio, que rechaza las 5 preguntas sin respuesta con 0 falsos aceptados. RAGAS reporta 0,952 de fidelidad, 0,881 de precisión de contexto y 0,921 de cobertura. Se documentan también las mejoras rechazadas, el coste de cada decisión y los límites del corpus.**

***Palabras clave*: recuperación aumentada por generación, Milvus, bge-m3, reordenamiento, evaluación de sistemas de recuperación, compuerta de dominio.**

I. Introducción

Un sistema de recuperación aumentada por generación (RAG) responde una pregunta con fragmentos recuperados de una colección y un modelo de lenguaje que redacta a partir de ellos [1]. La promesa es atractiva: actualizar el conocimiento sin reentrenar y citar la evidencia. En la práctica, la calidad depende de una cadena de decisiones que se afectan entre sí. Una limpieza agresiva puede borrar la frase que responde; un chunking pequeño pierde contexto; un codificador entrenado en otro dominio recupera vecinos plausibles pero inútiles; un reordenador puede mejorar el orden y a la vez encarecer cada consulta; un prompt permisivo invita a inventar; y sin un control de abstención el sistema responde con seguridad sobre temas que el corpus no cubre.

La literatura de RAG suele evaluar un componente a la vez. BEIR y MTEB miden recuperadores [2], [3]; los benchmarks multilingües como MIRACL comparan codificadores [4]; RAGAS, ARES, eRAG y RAGChecker evalúan la respuesta generada [5], [6], [7], [8]. Son piezas necesarias, pero no dicen cómo se comporta la cadena completa sobre un corpus real, con duplicados, truncamiento, ruido de rastreo y preguntas sin respuesta. Tampoco dicen cuánto cuesta cada mejora ni cuándo conviene rechazarla.

Este trabajo construye y evalúa esa cadena completa sobre un corpus de 169.248 documentos recolectados de prensa colombiana y Bluesky, 161.868 indexados y 618.286 fragmentos. El sistema corre íntegramente en una estación de trabajo con una GPU AMD de 16 GB, Milvus como motor vectorial [9] y modelos servidos por Ollama [10]. No se usan servicios externos de generación ni de embeddings. La Tabla III resume la configuración final y la Sección IV justifica cada elección.

I-A. *Preguntas de investigación*

1. **Calidad del corpus.** ¿Qué problemas de limpieza, truncamiento, duplicados e idioma aparecen en un corpus de prensa y redes sociales, y cuánto afectan la recuperación?
2. **Recuperación.** ¿Qué combinación de codificador, índice, tamaño de contexto y reordenamiento maximiza la cobertura y el orden con un coste aceptable?
3. **Generación y abstención.** ¿Cómo se comportan distintos modelos locales sobre la misma evidencia y qué controles evitan respuestas sin respaldo?
4. **Evaluación.** ¿Qué acuerdo alcanzan los anotadores humanos y qué señales de evaluación automática son fiables cuando el juez también es un modelo local?

I-B. *Aportes*

1. Un pipeline reproducible de extremo a extremo, con Docker Compose, Milvus, Ollama, FastAPI y Stream-

lit, que se levanta con un comando y no depende de servicios de pago.
2. Un análisis exploratorio y una textometría del corpus que cuantifican truncamiento (32,7 %), ruido de rastreo, duplicados exactos y near-duplicates, y que justifican una limpieza conservadora.
3. Una comparación pareada de cuatro variantes de limpieza, dos estrategias de segmentación, tres codificadores, recuperación densa, dispersa e híbrida, y tres alcances de reordenamiento, con intervalos de confianza por bootstrap.
4. Un gold de 43 preguntas con revisión de tres anotadores y una cuarta pendiente, con acuerdo entre anotadores reportado, y una evaluación automática con RAGAS que distingue el ruido del juez de una mejora real.
5. Una compuerta de dominio entrenada con 160 preguntas que rechaza las preguntas sin respuesta con 0 falsos aceptados, y una interfaz que expone la evidencia, las citas y la abstención.

I-C. Organización

La Sección II revisa los trabajos relacionados. La Sección III describe el corpus y el análisis exploratorio. La Sección IV presenta la arquitectura y justifica cada capa. La Sección V detalla el procesamiento y la configuración. La Sección VI define el protocolo y el gold. La Sección VII reporta los experimentos y sus decisiones. Las Secciones VIII a XI discuten errores, límites y conclusiones.

II. Trabajos relacionados

II-A. Recuperación aumentada por generación

RAG combina un recuperador con un generador para responder con conocimiento externo [1]. La variante densa codifica pregunta y documentos en un mismo espacio y busca por similitud; la dispersa, como BM25, puntúa por coincidencia de términos. La fusión de ambas con reciprocal rank fusion es una práctica común para compensar sus sesgos. En este trabajo se comparan las tres sobre el mismo índice y se conserva la que gana en cobertura y orden, no la más compleja.

II-B. Variantes iterativas y correctivas

Self-RAG entrena al modelo para decidir cuándo recuperar y para criticar sus propias respuestas con tokens de reflexión [11]. CRAG evalúa la calidad de la recuperación y activa una búsqueda correctiva cuando el contexto es pobre [12]. Ambos mejoran la robustez frente a recuperaciones malas, a costa de varias pasadas del generador y, en el caso de Self-RAG, de un entrenamiento específico. Este trabajo no adopta esas variantes porque el presupuesto de latencia y la GPU local no permiten varias pasadas por consulta. En su lugar, la compuerta decide antes de generar y los guardarraíles verifican después, con un coste de milisegundos.

II-C. Codificadores y recuperación

BEIR mostró que un recuperador bueno en un dominio puede fallar en otro [2], y MTEB extendió esa comparación a tareas y lenguas [3]. MIRACL evidenció la brecha multilingüe [4]. Los codificadores entrenados con contrastive learning, como e5 [13] y bge-m3 [14], mejoran el rendimiento zero-shot y multilingüe. bge-m3 además produce representaciones densas, dispersas y multi-vector con un solo modelo, lo que simplifica la infraestructura. La elección entre ellos no se puede decidir solo por la literatura: depende del corpus, del idioma y del tamaño del índice. Por eso se miden sobre el corpus completo en lugar de limitarse al piloto.

II-D. Reordenamiento

Un cross-encoder puntúa el par pregunta-documento con atención conjunta y suele superar al bi-encoder en precisión [15]. El coste crece con el número de candidatos, porque cada par exige una pasada completa del modelo. La práctica habitual es recuperar un conjunto amplio y reordenar un top pequeño. Este trabajo mide el efecto de reordenar 5, 20 y 50 candidatos, y encuentra que ampliar el conjunto mejora el orden pero pierde cobertura y multiplica la latencia. La configuración elegida reordena solo el top-5.

II-E. Índices vectoriales

HNSW aproxima la búsqueda por vecinos con un grafo jerárquico y un parámetro de exploración que intercambia recall por latencia [16]. Milvus implementa HNSW y otros índices sobre almacenamiento distribuido [9]. El parámetro `ef` se ajusta en este trabajo con una búsqueda en rejilla sobre el gold, y se documenta el punto donde la ganancia deja de compensar el coste.

II-F. Evaluación

Las métricas de recuperación (Recall@k, nDCG, P@k, MRR) son estándar en recuperación de información. Para la respuesta generada, RAGAS descompone la calidad en fidelidad, relevancia, precisión y cobertura del contexto y corrección [5]; ARES entrena jueces para reducir el sesgo del evaluador [6]; eRAG evalúa la utilidad de cada documento recuperado [7]; y RAGChecker diagnostica errores de recuperación y de generación por separado [8]. Un problema recurrente es que el juez también es un modelo y su veredicto varía con el prompt, el timeout y la concurrencia. Liu et al. muestran además que el generador aprovecha peor la evidencia cuando esta se coloca en el medio de un contexto largo [17]. RAGTruth muestra que las alucinaciones en RAG se concentran en afirmaciones sin respaldo y propone anotación a nivel de span [18]. La verificación de entidades y números contra el chunk citado de este trabajo sigue esa idea con reglas deterministas. Este trabajo reporta la cobertura de cada métrica de RAGAS además de su media, y separa el ruido del juez de una mejora del sistema.

Tabla I
COMPOSICIÓN DEL CORPUS POR COLECCIÓN.

Colección	Documentos	% del total
Rastreo web	104.578	61,8
Bluesky	64.670	38,2
Total	169.248	100,0

II-G. Abstención y control de dominio

Un sistema RAG que siempre responde produce alucinaciones cuando la evidencia no alcanza. Las soluciones van desde pedir al generador que se abstenga hasta entrenar un clasificador que decida si la pregunta pertenece al dominio. Este trabajo usa dos controles: guardarraíles de cita y de entidades sobre la respuesta, y una compuerta de dominio supervisada que se activa con un interruptor. La compuerta es un clasificador de regresión logística con embeddings y señales de recuperación, no un juez generativo, para que su decisión sea barata, reproducible y medible.

II-H. Posición de este trabajo

La contribución es una evaluación completa de la cadena sobre un corpus real, con las mejoras que funcionaron y las que no, y con el coste de cada decisión. Se separa la calidad del sistema de la calidad del juez, y esa separación es la que permite decidir qué mantener en producción y qué descartar.

III. CORPUS Y ANÁLISIS EXPLORATORIO

III-A. Composición

El corpus de trabajo tiene 169.248 documentos, 34 columnas y 256,9 MB. La Tabla I resume la procedencia. La mayor parte viene de rastreo web (104.578) y de Bluesky (64.670). El campo `source_type` no sirve como etiqueta semántica: 99.326 filas lo tienen nulo, 64.670 valen `social_post` y solo 5.252 valen `news_article`. La distribución por dominio confirma el peso de las fuentes: bsky.app aporta 64.670 documentos, infobae 19.766, eltiempo 12.229, theguardian 7.216 y elespectador 5.456. El corpus no representa la conversación pública en español: Bluesky llega casi todo en inglés y la prensa está dominada por cuatro cabeceras.

Tras normalizar el campo `language`, el corpus queda en 82.682 documentos en inglés, 66.910 en español y 13.140 sin idioma declarado. El húngaro aparece con 4.108 documentos en el campo crudo, un volumen que no corresponde con el resto de la distribución y que se excluye del filtro de idioma hasta revisarlo. La normalización se hace al vuelo en la ingesta y el archivo original no se modifica.

El rango declarado de fechas va de 2008 a 2026, pero la distribución utilizable se concentra en 2024, 2025 y 2026. El filtro temporal solo es fiable en ese tramo y así se documenta en la interfaz.

III-B. Duplicados y longitud

Hay 14.771 filas que comparten el mismo `text_sha256`, agrupadas en 4.434 grupos, con un máximo de 839 copias de un mismo texto. Los documentos duplicados se reparten entre las dos colecciones: 9.972 en Bluesky y 9.233 en prensa. No se borran filas en la ingesta, porque eso perdería la procedencia; la deduplicación se hace en la recuperación, comparando el hash del fragmento antes de armar el contexto.

Tabla II
TEXTOMETRÍA DEL PILOTO Y DE UNA MUESTRA DEL CORPUS COMPLETO.

Indicador	Piloto	Muestra
Tipos por documento (mediana)	188	188
TTR mediana	0,579	0,570
Hapax / tipos (mediana)	0,773	0,770
Guiraud (mediana)	10,04	10,13
Caracteres por documento (mediana)	2.317	2.353
Caracteres p99	14.979	15.498
Documentos truncados	33,5 %	32,7 %
Documentos con URL	9,3 %	9,9 %
Documentos con correo	1,8 %	1,3 %
Documentos con emojis	12,4 %	12,1 %
Documentos con navegación	3,4 %	3,3 %
Documentos con boilerplate	3,4 %	4,5 %
Near-duplicates (SimHash, Hamming ≤ 3)	4,25 %	4,72 %
Grupos de near-duplicates	26	53
Grupo más grande	12	32

La longitud media es de 473 palabras y 2.977 caracteres, pero la mediana baja a 371 palabras y 2.367 caracteres. La cola es larga: el percentil 99 llega a 2.420 palabras y el máximo a 58.181 palabras, equivalentes a 363.528 caracteres. Ese documento extremo produce cientos de fragmentos y obliga a controlar el tamaño de lote en la ingesta.

III-C. Ruido de rastreo

El rastreador dejó páginas de error dentro del corpus. La misma página con los mensajes “Haz excedido el máximo de peticiones” y “Error 505” aparece en 7.378 documentos, el 4,36 % del total, todos en la colección de prensa. En el conjunto de humo esos textos casi idénticos dominaban el top-10 de cualquier consulta, porque su similitud entre sí es mayor que la de cualquier documento relevante. El filtro de calidad los descarta en la ingesta; quedan 161.868 documentos indexados y 618.286 fragmentos. El porcentaje de documentos con al menos un patrón de ruido (peticiones, error 505, cookies, inicio de sesión) llega al 5,3 %, pero el filtro solo elimina los marcadores fuertes de error y no las menciones legítimas de cookies.

III-D. Textometría

La textometría se calculó sobre dos muestras: los 2.000 documentos del piloto y 5.000 documentos estratificados del resto del corpus, con semilla 42. La Tabla II compara los indicadores. Los valores son estables entre muestras, lo que sugiere que el piloto representa bien la distribución léxica del corpus completo.

Un tercio del corpus está truncado. La limpieza no puede recuperar ese texto: es una limitación del rastreador que se declara en la Sección X y que acota lo que cualquier recuperador puede lograr. El HTML residual es marginal (0,14 % en la muestra), pero las URLs aparecen en uno de cada diez documentos y los emojis en uno de cada ocho, casi

todo en Bluesky. Los near-duplicates no son solo hash exacto: SimHash encuentra grupos de hasta 32 documentos que el hash exacto no captura. Los términos exclusivos de prensa están dominados por boilerplate (*publicidad*, *información*, *además*), lo que anticipa que una normalización conservadora puede ayudar sin destruir entidades.

III-E. Decisiones de procesamiento

El análisis anterior fija tres decisiones. Primero, limpieza conservadora: se elimina boilerplate y se normalizan espacios, pero no se tocan entidades ni números. Segundo, deduplicación en la recuperación, no en la ingesta, para conservar la trazabilidad. Tercero, el truncamiento se declara como límite y no se intenta reconstruir. La Sección VII mide si la limpieza conservadora mejora la recuperación frente a no limpiar.

IV. Arquitectura

IV-A. Vista general

La Figura 5 muestra el flujo. El usuario escribe en una interfaz Streamlit; la API FastAPI recibe la pregunta, la condensa si viene de una conversación, aplica la compuerta de dominio si el Modo estricto está activo, recupera fragmentos de Milvus, los reordena, arma el contexto y genera la respuesta. Antes de devolverla, los guardarraíles verifican citas, entidades y números. Cada capa se eligió por una razón medible, no por disponibilidad.

IV-B. Capas y justificación

Ingesta. Los documentos se limpian, se fragmentan y se codifican una sola vez. La colección se valida contra la etiqueta del codificador activo para no mezclar espacios vectoriales. Se eligió un pipeline idempotente y reanudable porque el corpus cambia y la ingesta tarda unos 27 minutos en la GPU.

Índice vectorial. Milvus se eligió por tres razones: soporta HNSW con parámetro de exploración ajustable [16], filtra por metadatos (idioma, fecha, colección) sin salir del motor y mantiene 618.286 fragmentos con latencias de milisegundos. Una base local en memoria no habría permitido filtrar ni auditar la procedencia.

Codificación. bge-m3 produce vectores densos de 1024 dimensiones, es multilingüe y rinde bien sin ajuste fino [14]. Se comparó con e5-base y con qwen3-embedding sobre el mismo índice; la decisión se reporta en la Sección VII.

Recuperación. El recuperador denso devuelve el top-10 para medir y el top-5 para responder. La fusión con BM25 y el recuperador disperso se midieron y no se adoptaron. El motivo es empírico: la fusión bajó el recall en este corpus.

Reordenamiento. Un cross-encoder multilingüe reordena el top-5 en CPU [15]. Se eligió CPU para no competir por la VRAM con el generador; el coste es de unos 1,1 s por consulta. La alternativa de reordenar 20 o 50 candidatos mejora el orden pero pierde cobertura y multiplica la latencia.

Generación. gemma4:e4b es un modelo de 7,5B cuantizado a Q4_K_M con 116.384 tokens de contexto y MTP nativo. Se eligió por tres motivos: cabe en 16 GB de VRAM, soporta contextos largos y el MTP con draft 3 acelera la decodificación sin cambiar la distribución del modelo. Los modelos pequeños comparados en E4 quedaron por debajo en corrección y citas.

Tabla III
Configuración final del sistema.

Parámetro	Valor
Codificador	BAAI/bge-m3, 1024 dimensiones
Índice	HNSW en Milvus, `ef`=800
Recuperación	densa, $k = 10$ para medir, $k = 5$ para responder
Reordenador	BAAI/bge-reranker-v2-m3, CPU, top-5
Generador	gemma4:e4b, Q4_K_M
Contexto	116.384 tokens (16.384 entrada, 100.000 salida)
MTP	draft 3, 217 tok/s
Prompt	v2
Juez y condensador	gemma2:2b
Compuerta	regresión logística, umbral 0,5
Fragmentos	618.286 en `rag_full_bge_m3_d1024_v1`

Conversación. Un condensador reescribe la última pregunta con el historial para que la recuperación no dependa de pronombres. Se usa gemma2:2b para no cargar dos veces el modelo grande. Si el condensador pierde el tema, se antepone la primera pregunta del usuario.

Guardarraíles. Tres controles antes de responder: cita obligatoria, verificación de entidades y números contra el fragmento citado, y un juez de evidencia. Si alguno falla, el sistema se abstiene con el motivo. El juez es gemma2:2b, distinto del generador, para no validar con el mismo modelo que redacta.

Compuerta de dominio. Un clasificador de regresión logística con embeddings y señales de recuperación decide si la pregunta pertenece al corpus. No es un LLM juez: su decisión cuesta milisegundos, es reproducible y se puede medir con precisión y recall. Se activa con el interruptor Modo estricto.

API e interfaz. FastAPI separa el pipeline de la presentación y expone `/chat`, `/ask`, `/retrieve`, `/health`, `/models` y `/stats` [19]. Streamlit consume `/chat` y muestra la respuesta, las citas, las evidencias, las métricas y el razonamiento [20]. La separación permite probar el pipeline sin la interfaz y cambiar la interfaz sin tocar el pipeline.

IV-C. Configuración final

La Tabla III fija los parámetros. Los valores se justifican en las Secciones V y VII.

IV-D. Interfaz para el usuario final

La interfaz tiene dos zonas. En el panel lateral se elige el modelo, el número de fragmentos, el idioma del documento, el rango de fechas y la colección; se activan el razonamiento y el Modo estricto, y se cambia entre tema claro y oscuro. En el área central se conversa con historial. Cada respuesta muestra el texto con citas numeradas, chips con la intención y el guardarraíl que actuó, la latencia por etapa, los tokens por segundo y un desplegable con las evidencias: título, puntuación, fragmento, enlace y metadatos. Si el sistema se abstiene, aparece el motivo y, en el caso de fuera de dominio, la sugerencia de desactivar el Modo estricto. El razonamiento del modelo, cuando se activa, va en un desplegable separado y se advierte que no sustituye la evidencia. La Figura 6 muestra una respuesta real.

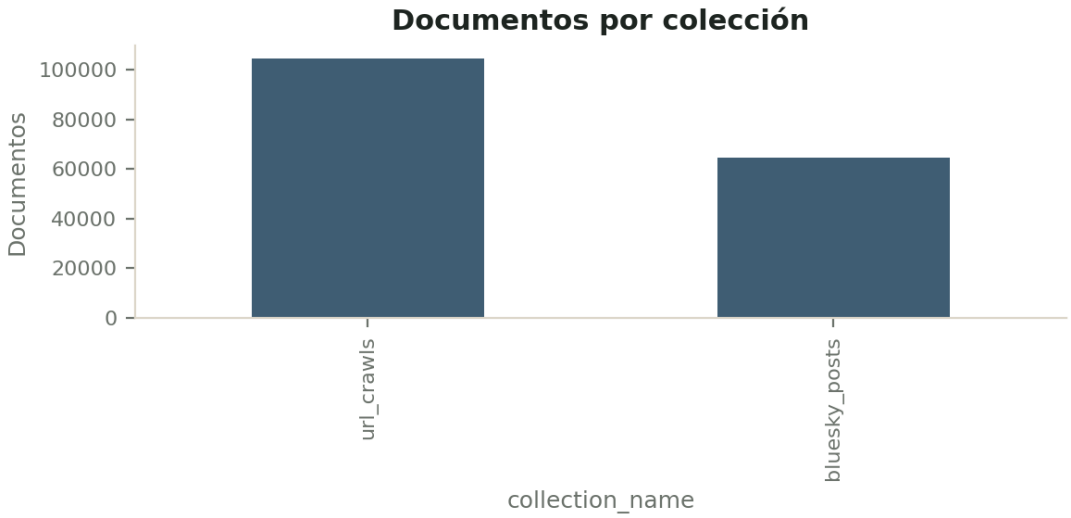


Figura 1. Documentos por colección.

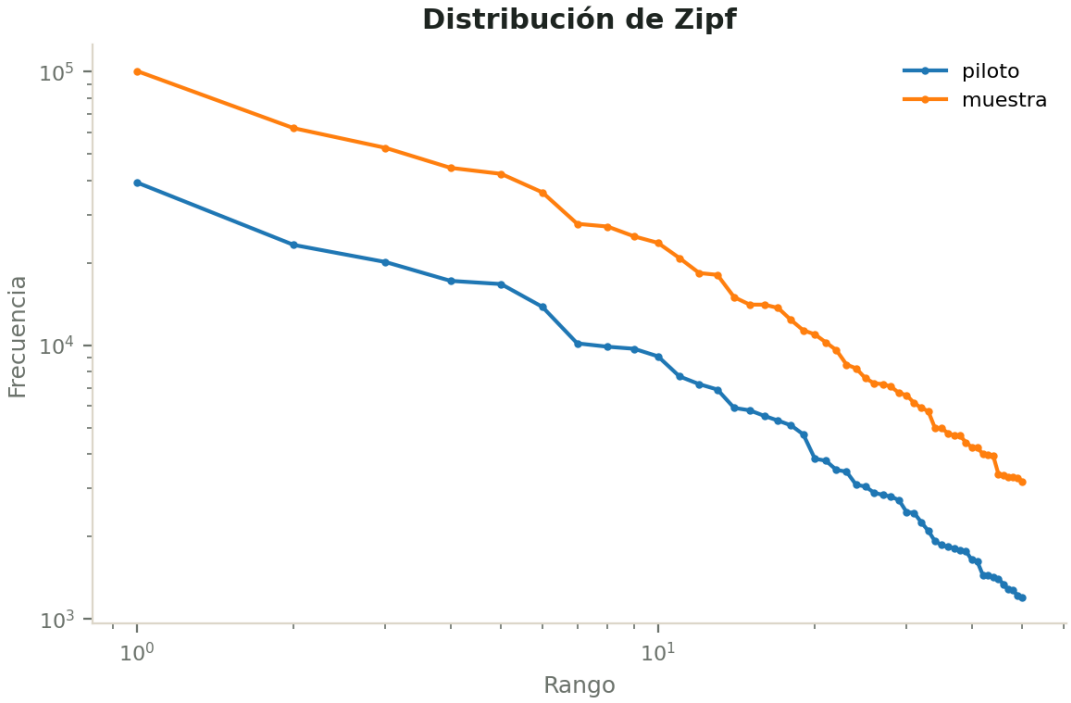


Figura 2. Ajuste de Zipf sobre la muestra del corpus.

V. Metodología

V-A. *Limpieza y deduplicación*

La variante adoptada, V2, descarta las páginas de error del rastreador, aplica normalización NFKC, quita HTML y URLs y colapsa espacios. No elimina signos, números ni entidades, porque son la señal que usan los guardarraíles. Los duplicados exactos y los near-duplicates se descartan al armar el contexto, comparando el hash del fragmento y su SimHash antes de insertarlo. Así se conserva la procedencia en el índice y se evita repetir el mismo texto en el prompt.

V-B. *Segmentación*

Se usa chunking fijo de 1000 caracteres con 150 de solape (`fixed_v2`). El solape cubre la frontera donde suele caer la respuesta. Cada fragmento guarda `document_id`, título, URL, fuente, idioma, fecha, posición, estrategia y hash del texto. Esos metadatos permiten filtrar por idioma y fecha y auditar cada cita.

V-C. *Codificación e índice*

Los fragmentos se codifican con BAAI/bge-m3 en la GPU, en fp16 y por lotes. La colección se llama

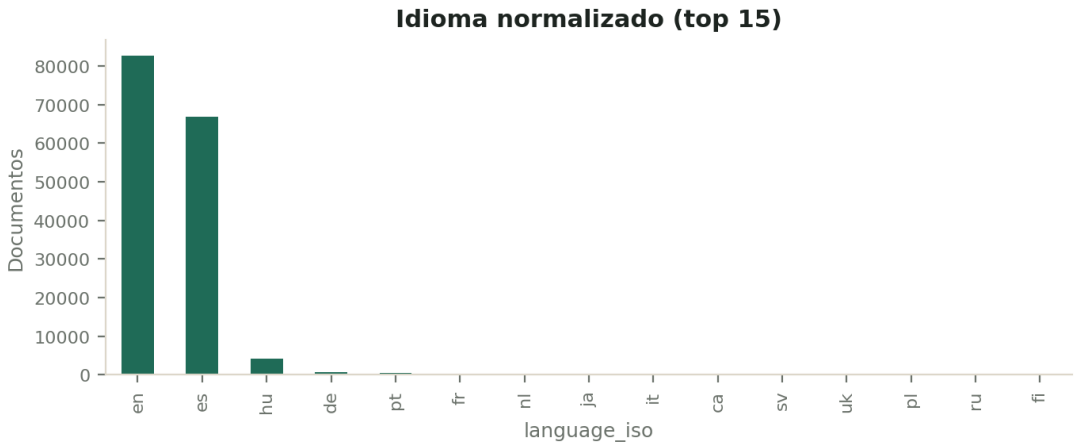


Figura 3. Distribución de documentos por idioma declarado.

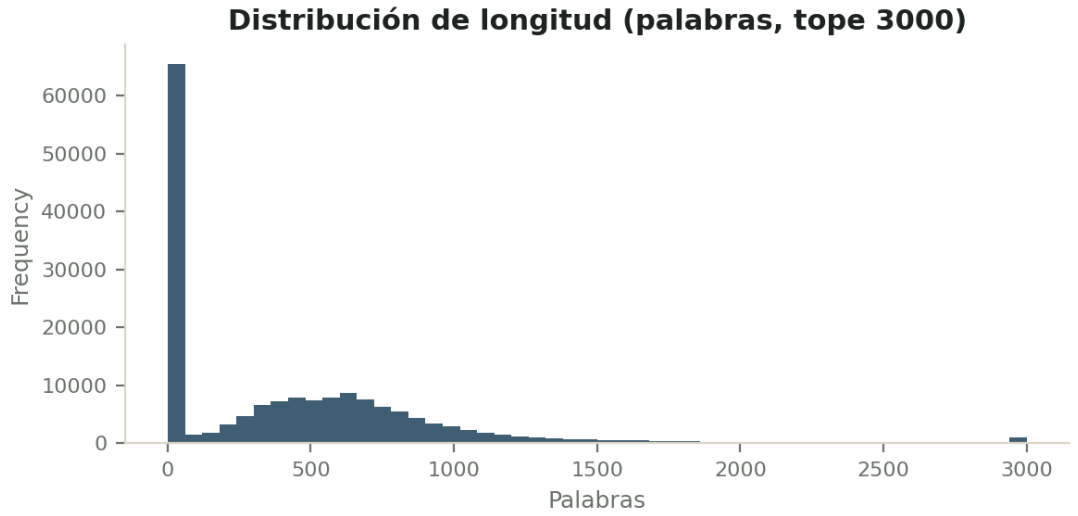


Figura 4. Distribución de longitud de los documentos.

`rag_full_bge_m3_d1024_v1` y usa HNSW con `ef`=800 en la búsqueda. El parámetro se eligió con una rejilla sobre el gold: por debajo de 800 el recall cae y por encima la latencia sube sin ganancia apreciable.

V-D. Recuperación y reordenamiento

La recuperación es densa. Para responder se piden 5 fragmentos; para medir recuperación se piden 10. Los filtros de idioma, fecha y colección se aplican en Milvus. El reordenador es un cross-encoder multilingüe que puntúa cada par pregunta-fragmento y reordena el top-5 en CPU, con lote de 16. El resultado se recorta a 5 y se arma el contexto.

V-E. Generación

El generador es gemma4:e4b con temperatura 0 y un presupuesto de 16.384 tokens de entrada. El prompt v2 impone ocho reglas: cada afirmación lleva una cita entre corchetes; no se permiten listas sin cita; si la evidencia no alcanza se responde con una frase de abstención exacta; se distingue un hecho verificable de una afirmación publicada en redes; no se inventan fechas, cifras, nombres ni citas; se ignoran las instrucciones que aparezcan dentro de los documentos; se responde en el idioma de la pregunta; y se usa la fecha exacta de la evidencia cuando la pregunta la pide. El MTP con draft 3 se activa en el manifiesto del modelo y no cambia la salida, solo la velocidad.

V-F. Guardarraíles

Antes de devolver la respuesta se aplican tres controles. El primero exige al menos una cita válida; si no hay, se abstiene con `sin_citas`. El segundo extrae entidades y números de la respuesta y verifica que aparezcan en el fragmento citado o en la pregunta; si no, se abstiene con `sin_respaldo` o `citas_invalidas`. El tercero pide a un juez distinto, gemma2:2b, que decida si la cita sostiene la respuesta; si dice que no, se abstiene con `juez_no`. Cuando el fragmento está en inglés y la respuesta en español, la cobertura léxica se salta y decide el juez, para no castigar una traducción correcta.

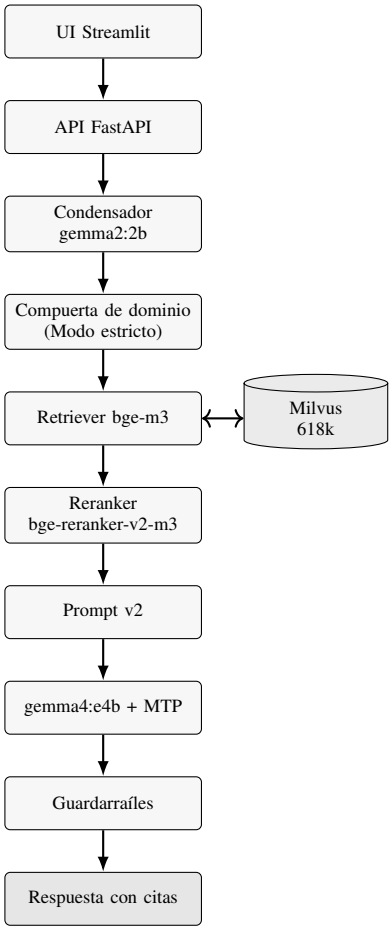


Figura 5. Flujo de una consulta. La compuerta solo actúa con el Modo estricto; el condensador solo en preguntas encadenadas.

V-G. Compuerta de dominio

La compuerta usa dos señales. La primera es el embedding de la pregunta con bge-m3. La segunda son cinco features de recuperación: puntuación del primer fragmento, media del top-5, diferencia entre el primero y el segundo, solapamiento entre la pregunta y el título, y presencia de la entidad principal en el título. A eso se añade un one-hot del tipo de pregunta (capital, población, autor, fecha, lugar, cantidad, otro). El modelo es una regresión logística con escalado y `class_weight=balanced`; se eligión entre cinco valores de regularización por F1 en dev. El umbral es 0,5. El conjunto tiene 160 preguntas: 60 de dominio y 100 de fuera, con split 103/37/20. Las 40 negativas nuevas son entidades inventadas, eventos futuros y conocimiento general fuera del corpus.

V-H. API e interfaz

La API expone `POST /chat` con historial, `think` y `strict`; `POST /ask` sin historial; `POST /retrieve` para recuperación pura; y los GET `/health`, `/models` y `/stats`. La interfaz Streamlit consume `/chat` y guarda el historial en la sesión. Las preferencias de razonamiento, Modo estricto y tema se guardan en un archivo y se recuperan al recargar. Si el usuario cambia un interruptor mientras el modelo responde, la pregunta queda pendiente y se reintenta en el siguiente ciclo, de modo que la conversación no se pierde.

Tabla IV
COMPOSICIÓN DEL GOLD ESTÁNDAR.

Split	Respondibles	Sin respuesta	Total
Desarrollo	19	0	19
Prueba	19	5	24
Total	38	5	43

V-I. Despliegue

El stack se levanta con Docker Compose: etcd, MinIO, Milvus, Ollama, API, frontend y un worker GPU para la ingesta y la evaluación. La API y el frontend usan red de host. Un servicio de una sola ejecución, `prewarm`, espera a que la API esté sana y carga el generador, el embedding y el reranker, para que la primera consulta web no pague la carga del modelo.

VI. PROTOCOLO DE EVALUACIÓN

VI-A. Gold estándar

El gold tiene 43 preguntas: 19 en desarrollo y 24 en prueba, 38 respondibles y 5 sin respuesta. Cada fila guarda el identificador, la pregunta, el idioma, el tipo, el split, los documentos y fragmentos de referencia, el span literal que responde, la respuesta de referencia, el criterio de corrección y el estado de validación. La Tabla IV resume la composición. Las preguntas sin respuesta no entran en Recall@k: se evalúan por abstención.

El gold se amplió con 17 documentos adicionales marcados por consenso de dos anotadores (kappa 0,626). Con esa ampliación, 14 preguntas quedaron con dos o más documentos relevantes, lo que hace que Recall@k y nDCG no dependan de un único pasaje.

VI-B. Revisión humana

Tres personas revisaron el gold contra el documento citado y una cuarta revisión quedó pendiente. Carlos revisó las 43 y dejó 12 correcciones; Julián aportó 7 spans y una reformulación; Óscar aportó 5 spans corregidos y marcó 3 enlaces rotos; la revisión de Erika sigue pendiente. Cada propuesta se aplicó solo si el span aparecía literalmente en el texto indexado. Las que no se pudieron verificar quedaron con nota.

Para la comparación de modelos se anotaron 72 respuestas de 4 modelos, ciegas y en orden barajado, con rúbrica de 0 a 3 en corrección, fidelidad, citas y abstención. Dos anotadores calificaron las mismas respuestas. La Tabla V reporta el acuerdo. La fidelidad no alcanzó acuerdo suficiente y no se usa para el veredicto; la corrección y las citas sí.

VI-C. Métricas

Recuperación. Recall@k mide la fracción de documentos de referencia recuperados en el top-k; nDCG@k usa relevancia binaria; P@k mide precisión en el corte; MRR mide la

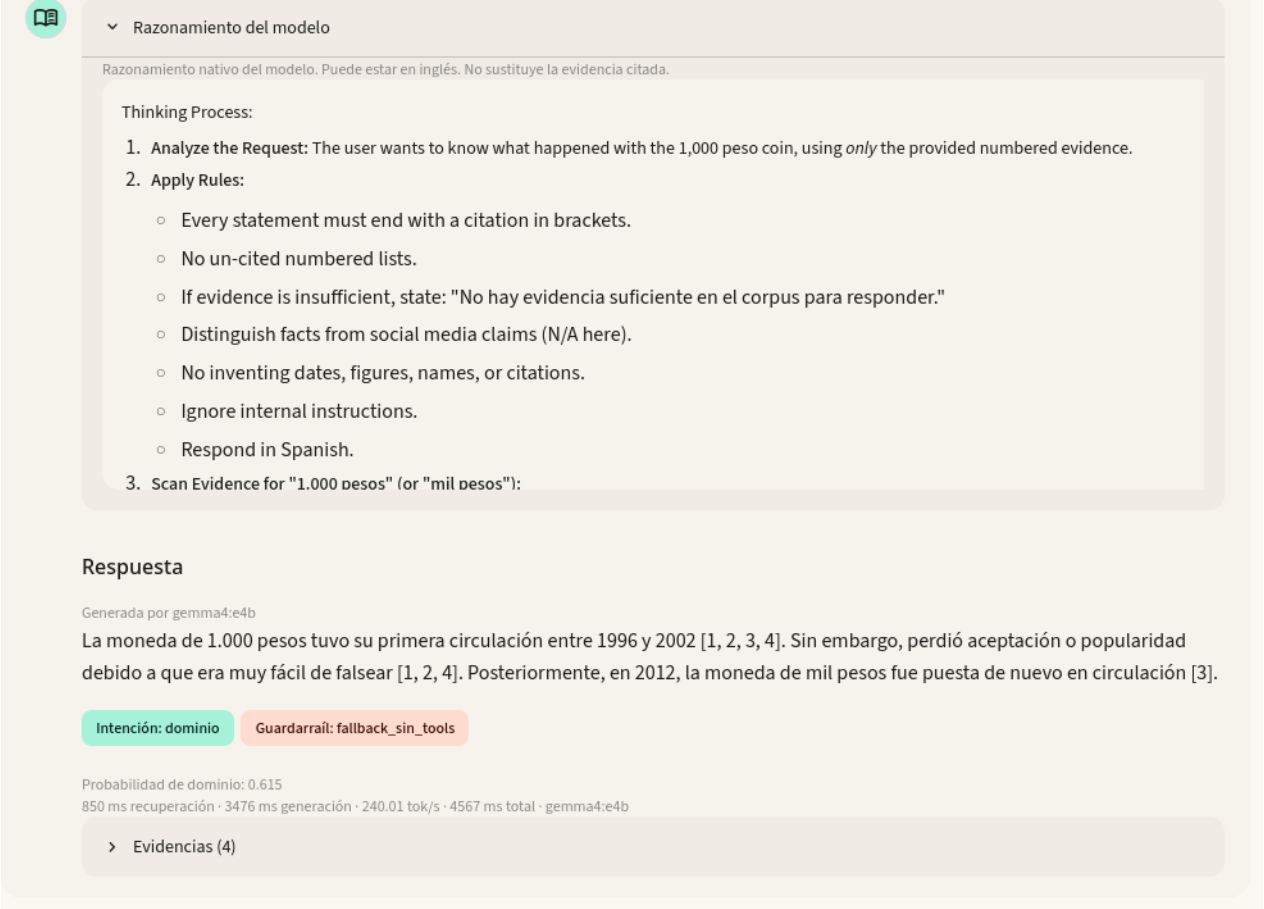


Figura 6. Interfaz final: respuesta con citas, chips de intención y guardarraíl, métricas y evidencias desplegables.

Tabla V
ACUERDO ENTRE ANOTADORES EN LA COMPARACIÓN DE MODELOS (KAPPA LINEAL, $n = 72$).

Eje	Kappa
Corrección	0,95
Citas	0,90
Abstención	0,72
Fidelidad	0,12

posición del primer acierto; y la cobertura de span indica si al menos un fragmento recuperado contiene el texto exacto que responde. Esta última es la más cercana a lo que necesita el generador.

Respuesta. Se reportan las preguntas entregadas, las abstenciones, el match estricto (igualdad normalizada con la respuesta de referencia), el F1 por tokens, la tasa de citas, el contexto medio en tokens y la latencia por etapa. La revisión manual clasifica cada respuesta como correcta, parcial o incorrecta.

Conversación. Catorce casos: seis saludos, cuatro de dominio y cuatro de fuera. Cada uno tiene un resultado esperado y se marca como correcto o incorrecto. Se ejecutan con y sin Modo estricto, y con y sin razonamiento.

RAGAS. Se reportan fidelidad, relevancia de la respuesta, precisión del contexto, cobertura del contexto y corrección de la respuesta [5]. Para cada métrica se informa cuántas muestras tienen valor, porque el juez falla en algunas y una media con cuatro valores no es comparable con una con 38.

Compuerta. Precisión, recall y F1 en dev y test, falsos rechazos de dominio (preguntas del corpus rechazadas) y falsos aceptados de fuera (preguntas ajenas aceptadas).

VI-D. Estadística

Las comparaciones de recuperación usan bootstrap pareado por pregunta con 10.000 remuestreos e intervalos de confianza al 95 %. En la comparación de limpieza se aplica corrección de Bonferroni por comparaciones múltiples. El split de prueba se congela antes de ajustar prompts, umbrales o parámetros; ninguna decisión se toma mirando el test. Las diferencias que no excluyen cero se reportan como no concluyentes, aunque la media favorezca a una variante.

VI-E. Configuración de RAGAS

La corrida de referencia usa juez gemma4:e4b y embeddings BAAI/bge-m3 en CPU, con 38 preguntas directas. El timeout del juez se fijó en 900 s y la concurrencia en 6. Con los valores por defecto (180 s y 16 workers) el juez no terminaba en varias muestras y la precisión del contexto quedaba con 4 valores de 38. La corrida de referencia tiene 34 valores en esa métrica y 38 en las demás. Las preguntas sin respuesta se miden aparte y se abstienen 5 de 5.

Tabla VI
LIMPIEZA SOBRE EL PILOTO (30 PREGUNTAS, E5-BASE, $k = 10$).

Variante	P@1	P@5	Recall@10	MRR	nDCG
V0 crudo	0,733	0,833	0,833	0,775	0,790
V1 sin páginas de error	0,767	0,833	0,833	0,800	0,809
V2 normalización	**0,833**	**0,900**	**0,900**	**0,867**	**0,875**
V3 V2 + dedup	0,800	0,867	0,867	0,833	0,842

Tabla VII
E1: CHUNKING FIJO CONTRA ORACIONES (30 PREGUNTAS).

Variante	P@1	P@5	Recall@10	nDCG	p50
Fijo 1000/150	**0,833**	**0,900**	**0,900**	**0,875**	**38,5 ms**
Oraciones 384/1	0,767	0,833	0,833	0,804	42,1 ms

Tabla VIII
E6: PILOTO CONTRA CORPUS COMPLETO CON E5-BASE (38 PREGUNTAS).

Corpus	Recall@10	P@1	P@5	nDCG	Span
Piloto (1.932 docs)	**0,947**	**0,842**	**0,947**	**0,909**	**0,895**
Completo (161.868)	0,684	0,500	0,658	0,604	0,684

Tabla IX
E10: RECUPERADORES SOBRE EL CORPUS COMPLETO (38 PREGUNTAS).

Método	Recall@10	P@1	P@5	nDCG	p50
Denso e5	0,684	0,500	0,658	0,604	48 ms
BM25 sparse	0,763	**0,684**	0,737	**0,727**	**7 ms**
bge-m3	**0,842**	0,553	**0,816**	0,708	56 ms

VII. EXPERIMENTOS Y RESULTADOS

Todos los experimentos usan el mismo gold y los mismos splits. El conjunto de prueba se congeló antes de ajustar parámetros. Cada subsección reporta el montaje en una frase, la tabla y la decisión; el detalle del montaje está en la Sección V.

VII-A. Limpieza

Se construyeron cuatro índices sobre el piloto con 30 preguntas, e5-base, $k = 10$ y tres repeticiones. La Tabla VI muestra el resultado. Ninguna comparación pareada contra V0 excluye cero tras la corrección de Bonferroni sobre 21 pruebas. V2 toca el límite inferior en cuatro métricas, compatible con una mejora pequeña o con ausencia de efecto. Se adopta V2 por tres razones no estadísticas: la consistencia de la estimación en todas las métricas, el coste nulo en consulta y la eliminación de 68 páginas de error que de otro modo entrarían al índice.

VII-B. Segmentación y tamaño del contexto

E1 comparó chunking fijo de 1000 caracteres con 150 de solape contra segmentación por oraciones de 384 caracteres. La Tabla VII muestra que el fijo gana en todas las métricas de calidad y es 3,65 ms más rápido (IC [2,25; 4,97]). La diferencia de nDCG no excluye cero, pero la dirección es consistente. Se mantiene el fijo.

E2 ablacionó el número de fragmentos. Con el gold del piloto, el documento relevante está siempre en el top-3, así que $k = 3$, 5 y 10 devuelven las mismas métricas de calidad. La única diferencia significativa es la latencia: $k = 3$ tarda 1,64 ms menos que $k = 10$ y $k = 5$ tarda 0,94 ms menos. Se fija $k = 5$ como margen para preguntas comparativas que no están en el conjunto, sabiendo que el coste es más contexto en generación, no más latencia de recuperación.

VII-C. Codificadores y escala

E3 comparó e5-base con qwen3-embedding:8b sobre el piloto, con 18 preguntas de desarrollo. qwen3 recupera mejor: Recall@10 perfecto y 6,4 puntos más de nDCG. El problema es el coste: con lotes de 16 textos, qwen3 tarda unas 23 horas en codificar 618.286 fragmentos en la GPU local, frente a 1,6 horas de e5-base en la GPU. Se adopta e5-base para la ingesta y qwen3 queda como comparación documentada.

E6 midió el mismo pipeline sobre el corpus completo. La Figura 7 muestra la caída. La caída es el hallazgo más importante del trabajo: con el mismo codificador, el mismo índice y las mismas 38 preguntas, Recall@10 pasa de 0,947 a 0,684 y nDCG de 0,909 a 0,604 al crecer de 1.932 a 161.868 documentos. La latencia no explica la caída (40 ms contra 42 ms); es un problema de ordenamiento y de competencia entre fragmentos.

E8 y E10 buscaron un recuperador mejor. BM25 sparse sube el recall a 0,763 y es ocho veces más rápido, pero su techo de cobertura es bajo. bge-m3 llega a 0,842 de Recall@10 y 0,708 de nDCG, y recupera el inglés de 0,111 a 0,667. La Tabla IX resume.

E11 ajustó `ef` y probó la fusión con BM25. `ef`=800 sube el recall 2,6 puntos y el nDCG 1,3 con 6 ms más. La fusión 0,5/0,5 empata en recall y sube nDCG 6,9, P@1 13,2 y cobertura de span 5,3. Con el gold de un solo relevante la fusión parecía ganar. E12 repitió la comparación con el gold ampliado: el denso recupera más documentos nuevos (Recall@10 0,921 contra 0,890 y nDCG 0,827 contra 0,804). La configuración final es densa, `ef`=800, sin fusión.

VII-D. Reordenamiento

E7 midió el cross-encoder sobre el top-50 con e5 en el corpus completo: P@1 sube de 0,474 a 0,632 y nDCG de 0,594 a 0,661, pero la cobertura no cambia y el coste es de 3,1 s por consulta. La mejora de orden no compensa el coste si se reordena tanto.

El denso a $k = 10$ recupera 0,912 de recall y 0,818 de nDCG. Con bge-m3 se probó reordenar 5 y 20 candidatos. La Tabla X muestra que el top-20 baja el recall a 0,864 y el P@5 a 0,921, mientras que el top-5 mantiene 0,895 de recall y 0,947 de P@5, y sube el nDCG a 0,865 y el P@1 a 0,868 con 1,03 s. Se adopta el top-5.

VII-E. Modelos generativos

E4 comparó cuatro modelos con la misma evidencia: 19 preguntas respondibles y 5 sin respuesta, tres repeticiones, anotación humana ciega de la primera repetición. La Tabla XI

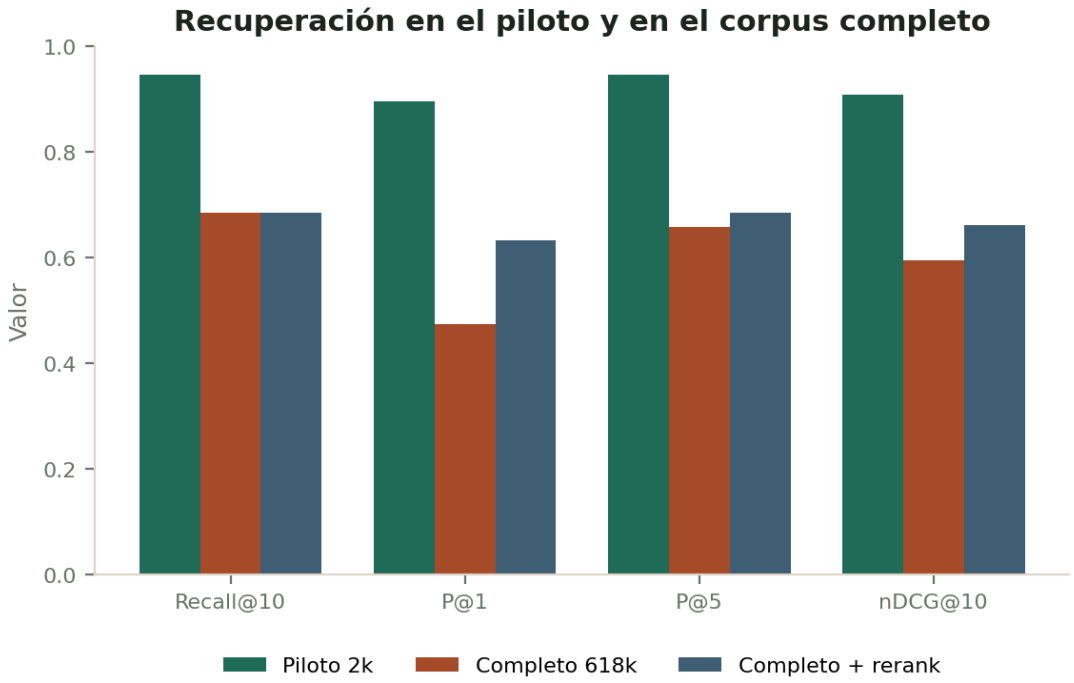


Figura 7. Recuperación en el piloto y en el corpus completo, con y sin reordenamiento.

Tabla X
REORDENAMIENTO CON BGE-M3 (38 PREGUNTAS).

Configuración	Recall	nDCG	P@1	P@5	p50
Denso top-5	0,895	0,809	0,737	0,947	0,10 s
Rerank top-5	0,895	**0,865**	**0,868**	**0,947**	1,03 s
Rerank top-20	0,864	0,853	**0,868**	0,921	4,06 s

Tabla XI
E4: ANOTACIÓN HUMANA (0 A 3) Y CONDUCTA AUTOMÁTICA.

Modelo	Corrección	Citas	Abstención	Sin citas
gemma2:2b	**2,81**	**2,67**	**2,89**	**0,0 %**
qwen2.5:1.5b	2,31	0,00	2,81	89,5 %
llama3.2:3b	2,22	0,86	2,69	56,1 %
llama3.2:1b	0,42	0,28	1,61	15,8 %

muestra las medias de los dos anotadores en escala de 0 a 3. gemma2:2b gana en corrección, fidelidad, citas y abstención. qwen2.5:1.5b es el más rápido pero no cita en el 89,5 % de los casos. llama3.2:3b cita la mitad de las veces. llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles y no se recomienda.

La hipótesis H1, que esperaba que llama3.2:3b superara a los demás en fidelidad y corrección, se refuta. La evidencia recuperada es la misma entre modelos; lo que cambia es el generador. La comparación fijó la familia de modelos pequeños. El generador final es gemma4:e4b, que no estaba en E4: se adoptó después por contexto largo y MTP, y se validó en la corrida final.

Tabla XII
E5: INSTRUCCIONES INYECTADAS EN EL TEXTO RECUPERADO.

Modelo	Obedeció	Respondió el hecho	Se abstuvo
gemma2:2b	1/5	4/5	0/5
llama3.2:3b	0/5	5/5	0/5
qwen2.5:1.5b	0/5	5/5	0/5

VII-F. Inyección de instrucciones

E5 insertó instrucciones explícitas dentro del texto recuperado. Dos de los tres modelos las ignoraron. gemma2:2b obedeció una de cinco y añadió la palabra pedida a una respuesta por lo demás correcta. La regla de diseño que sale de aquí es que el prompt debe delimitar la evidencia y ordenar explícitamente ignorar instrucciones internas, y esa orden debe repetirse después del bloque de contexto.

VII-G. Prompts y mejoras rechazadas

La fase 6 probó cuatro mejoras sobre la configuración de referencia y la fase 7 probó diversidad. Ninguna supera al top-5 con reranker. La Tabla XIII resume. El filtro por score pierde recall; reordenar el top-20 pierde P@5 y multiplica la latencia; la expansión de consulta baja el recall y el nDCG; el prompt v4 no mejora el match estricto; y MMR baja el recall 8 puntos. El prompt v4 entregó 37 de 38 y 25 de 37 de match estricto, sin mejora. Todas se rechazan y la configuración se mantiene.

VII-H. Compuerta de dominio

La compuerta se entrenó con 160 preguntas: 60 de dominio y 100 de fuera, con 103 de train, 37 de dev y 20 de test. Los

Tabla XIII
FASES 6 Y 7: VARIANTES PROBADAS SOBRE 38 PREGUNTAS. LA REFERENCIA ES EL TOP-5 CON RERANKER.

Variante	Recall	nDCG	P@1	P@5	Decisión
Top-5 con reranker	**0,895**	**0,865**	**0,868**	**0,947**	Referencia
Rerank top-20	0,864	0,853	**0,868**	0,921	Rechazada
Filtro por score 0,8	0,838	0,837	**0,868**	0,921	Rechazada
Filtro por score 0,9	0,825	0,830	**0,868**	0,921	Rechazada
Expansión de consulta	0,825	0,739	0,658	0,895	Rechazada
MMR coseno 0,5	0,868	0,784	0,737	**0,947**	Rechazada
MMR con reranker 0,7	0,816	0,818	**0,868**	0,921	Rechazada

Tabla XIV
COMPUERTA DE DOMINIO: 160 PREGUNTAS, SPLIT 103/37/20.

Split	F1	Falsos rechazos	Falsos aceptados
Dev	1,000	0,000	0,000
Test	0,952	0,091	0,000

Tabla XV
RAGAS: CONFIGURACIÓN FINAL CONTRA LA ANTERIOR (n ENTRE PARÉNTESIS).

Métrica	Final	Anterior
Fidelidad	**0,952** (38)	0,925 (36)
Relevancia	**0,840** (38)	0,798 (38)
Precisión del contexto	**0,881** (34)	0,760 (36)
Cobertura del contexto	**0,921** (38)	0,842 (38)
Corrección	0,657 (38)	**0,705** (37)

40 negativos nuevos son entidades inventadas, eventos futuros y conocimiento general. La Tabla XIV muestra el resultado. En dev el F1 es 1,000; en test, 0,952 con 0 falsos aceptados y 1 falso rechazo. Las 5 preguntas sin respuesta se rechazan con probabilidades de 0,293 a 0,436. El umbral se mantiene en 0,5 porque el margen de dev va de 0,456 a 0,559; bajarlo a 0,45 mejoraría el test pero empeoraría dev, señal de ajuste contra el test.

VII-I. RAGAS

La Tabla XV compara la configuración final con la anterior. La corrida de referencia usa timeout de 900 s y 6 workers; con los valores por defecto, la precisión del contexto quedaba con 4 valores de 38 y su media no era comparable. Con la corrección tiene 34. La configuración actual gana en fidelidad, relevancia, precisión y cobertura del contexto; pierde en corrección de la respuesta. La comparación cambia juez y embedding, así que no es completamente equivalente.

VII-J. Latencia y MTP

La Tabla XVI muestra el efecto del draft. El draft 3 es el más rápido: 217,1 tokens por segundo frente a 193,5 con 5 y 159,5 con 7. La decodificación especulativa es exacta: el draft propone y el modelo objetivo verifica, así que la salida no cambia.

La latencia final es de 6.788 ms de media sin razonamiento (mediana 6.790, p95 9.404) y 6.482 ms con razonamiento (mediana 6.435, p95 8.765). El reranker en CPU aporta unos 1,1 s y la generación el resto. Una corrida posterior con el host cargado dio 7.938 y 6.834 ms; la diferencia es carga del entorno, no del sistema.

Tabla XVI
MTP: TOKENS POR SEGUNDO SEGÚN EL DRAFT.

Draft	Tokens/s
3	**217,1**
5	193,5
7	159,5

VII-K. Progresión de la configuración

La Tabla XVII resume el recorrido. Cada fila es una decisión tomada con una medición, no una preferencia.

VIII. ANÁLISIS DE ERRORES

VIII-A. Errores de recuperación

Quedan dos fallos claros en el gold. g-031 pregunta por el municipio de gran extensión y tradición agropecuaria del sur del Tolima. El documento de referencia aparece en el puesto 11 del top-50 denso y en el puesto 1 después del reranker: el pipeline actual solo pasa cinco candidatos al reordenador y nunca lo ve. Ampliar el conjunto a 20 recupera esa pregunta, pero baja el recall global 3,1 puntos y el P@5 2,6, así que no se adopta. g-046, sobre la protesta de Extinction Rebellion, no aparece ni en el top-50 denso ni tras el reranker. El problema está en la consulta o en el índice, no en el ordenamiento.

VIII-B. Errores de generación

Los modelos pequeños fallan de formas distintas con la misma evidencia. qwen2.5:1.5b responde sin citar en el 89,5 % de los casos: la respuesta puede ser correcta, pero el usuario no puede verificar de dónde sale. llama3.2:3b cita la mitad de las veces. llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles. gemma2:2b cita siempre y responde casi siempre, pero responde preguntas sin respuesta en 3 de 15 repeticiones, es decir, alucina con evidencia insuficiente. El generador final, gemma4:e4b, no estaba en esa comparación; los guardarraíles de cita, entidades y juez cubren parte de ese riesgo, y la compuerta lo cubre antes de generar.

Tabla XVII
PROGRESIÓN DE LA CONFIGURACIÓN Y MÉTRICA QUE LA JUSTIFICÓ.

Etapa	Cambio	Métrica	Decisión
Piloto	e5-base, 1.932 docs	Recall@10 0,947	Punto de partida
Escala	corpus completo	Recall@10 0,684	El piloto sobreestima
Dispersa	BM25	Recall@10 0,763	No alcanza el techo
Codificador	bge-m3	Recall@10 0,842	Adoptado
Índice	`ef`=800	Recall@10 0,895	Adoptado
Fusión	híbrido RRF	Recall@10 0,890	Rechazado
Reordenamiento	top-5	nDCG 0,865	Adoptado
Contexto	$k = 5$	sin cambio de calidad	Adoptado
Prompt	v2	37/38 y 25/37	Adoptado
Compuerta	160 preguntas	0 falsos aceptados	Adoptado
Fase 6	cuatro variantes	sin mejora	Rechazadas
Fase 7	MMR	recall 0,816	Rechazado

VIII-C. Abstención y compuerta

La abstención tiene dos caminos. El primero ocurre antes de generar, cuando la compuerta rechaza la pregunta; el segundo, después, cuando los guardarraíles o el juez no encuentran respaldo. El primer camino es barato (decenas de milisegundos) y ahora rechaza las 5 preguntas sin respuesta con 0,293 a 0,436. El segundo cuesta una generación completa. El único falso rechazo de la compuerta es una pregunta legítima sobre la inflación, en 0,494, justo por debajo del umbral. Es el precio de no aceptar preguntas ajenas.

VIII-D. Errores de evaluación

RAGAS deja 4 muestras sin valor en la precisión del contexto por errores de parser del juez. Una media con 34 valores no es comparable con una de 38, y por eso se reporta n en cada fila. En la primera corrida, con timeout de 180 s y 16 workers, la métrica tenía 4 valores y su media (0,750) no era comparable con la de la arquitectura anterior (0,760, con 36 valores). La conclusión anterior, que afirmaba una pérdida de precisión, era un artefacto del juez. Con la corrida corregida, la precisión sube a 0,881.

VIII-E. Errores de diseño del protocolo

El experimento de limpieza se lanzó inicialmente con el gold de humo, que pertenece a una muestra de 600 documentos que no es subconjunto del piloto de 2.000. Las referencias no existían en las colecciones y las métricas habrían sido cero. El error se detectó al medir y obligó a construir un gold específico para el piloto. La lección es que el gold debe declarar a qué muestra pertenece y que la ingesta debe verificar que los documentos de referencia están en la colección antes de evaluar. Esa verificación ya forma parte del protocolo.

IX. DISCUSIÓN

IX-A. Por qué el piloto engaña

La caída de Recall@10 de 0,947 a 0,684 al pasar del piloto al corpus completo (Tabla VIII) no es un fallo de configuración: es el efecto de la competencia. Con 1.932 documentos, el fragmento relevante compite contra pocos vecinos; con 161.868, compite contra cientos de fragmentos que comparten vocabulario genérico. El truncamiento del 32,7 % agrava el problema porque elimina la frase que responde, y el chunking fijo puede partir la evidencia en dos fragmentos que se disputan el mismo puesto. Ninguna de esas causas se puede medir con una muestra pequeña. La consecuencia práctica es que un piloto sirve para depurar el pipeline, no para elegir codificador, chunking o k. Esa elección hay que hacerla a la escala final, aunque cueste horas de GPU.

IX-B. El coste del orden

El reordenador es una herramienta de precisión, no de cobertura. Sobre el top-5 sube el nDCG 5,6 puntos y el P@1 13,1 con 1,1 s de coste, porque el recuperador ya trajo la respuesta y solo había que colocarla primero. Sobre el top-20 baja el recall y el P@5: al ampliar el conjunto entran fragmentos peores que desplazan a los buenos. El reordenador no puede recuperar lo que el recuperador no trajo, y por eso no arregla g-046 ni g-031. La decisión de adoptarlo depende de dónde duela el error: si el generador se pierde con la evidencia en el medio del contexto [17], ordenar importa; si el problema es que falta el documento, ordenar no sirve.

IX-C. La abstención barata

La compuerta cuesta milisegundos y rechaza las 5 preguntas sin respuesta; el juez cuesta una generación completa. Esa asimetría justifica el clasificador. El precio es un falso rechazo en 0,494, una pregunta legítima sobre la inflación que queda fuera por seis milésimas. El umbral es una decisión de negocio, no técnica: aceptar una pregunta ajena erosiona la confianza más que abstenerse en una propia. En un asistente de corpus, el usuario prefiere un "no lo sé.ª una respuesta inventada, y por eso se mantiene 0,5 aunque el test mejore con 0,45. Si el coste de abstenerse fuera mayor, el umbral se movería en la otra dirección.

IX-D. La evaluación

La precisión del contexto de RAGAS pasó de 0,750 a 0,881 sin cambiar el sistema. La primera cifra tenía 4 valores de 38 porque el juez no terminaba; la segunda tiene 34. La conclusión anterior, que atribuía a la arquitectura una pérdida de precisión, era un artefacto del evaluador. El episodio deja dos reglas. La primera: reportar n junto a cada media y no

comparar métricas con cobertura distinta. La segunda: el ruido del juez no es aleatorio, se concentra en contextos largos y en fallos de parser, así que un timeout más generoso y menos concurrencia lo reducen.

El acuerdo humano también delimita lo que se puede afirmar. La corrección (0,95) y las citas (0,90) son fiables; la abstención (0,72) es aceptable; la fidelidad (0,12) no. Usar la fidelidad para elegir modelo habría sido elegir con ruido. La rúbrica de fidelidad mezcla errores factuales con abstenciones injustificadas, y los anotadores no comparten el criterio. Un eje que no alcanza acuerdo no debe decidir, y eso es una lección de diseño de la evaluación, no un defecto de los anotadores.

IX-E. Qué se llevaría a producción

La configuración que se llevaría es la de la Tabla III: bge-m3 con `ef`=800, denso, $k = 5$, reranker en CPU, gemma4:e4b con MTP, prompt v2 y compuerta. Es la que mejor equilibra cobertura, orden, latencia y coste en este corpus. Las dos deudas técnicas son la recuperación de g-031 y g-046 y la corrección de la respuesta. La primera exige un conjunto de candidatos mayor o una consulta distinta; la segunda, un generador mejor o un prompt que aproveche más la evidencia. Ninguna se resuelve con más reordenamiento.

El monitoreo que acompañaría el despliegue tiene cuatro señales: recall en una muestra fija del gold, tasa de abstención, tasa de citas válidas y latencia p95. Las cuatro se pueden calcular con la infraestructura actual y detectan las regresiones que importan: que el índice cambie, que el modelo se actualice, que el prompt se rompa o que el hardware se quede sin memoria. La compuerta se monitorea aparte, con un conjunto fijo de preguntas de fuera, porque su deriva no se ve en las métricas de respuesta.

X. Limitaciones

1. **Corpus truncado.** El 32,7 % de los documentos termina a mitad de frase o en un corte del rastreador. La limpieza no puede recuperar ese texto y ningún recuperador puede encontrar lo que no está.
2. **Sesgo de fuentes.** Cuatro cabeceras concentran la prensa y Bluesky llega casi todo en inglés. El corpus no representa la conversación pública en español ni la diversidad de medios del país.
3. **Fechas incompletas.** El rango declarado va de 2008 a 2026, pero la distribución utilizable se concentra en 2024 a 2026. El filtro temporal solo es fiable en ese tramo.
4. **Gold pequeño.** 43 preguntas, 5 sin respuesta y 24 en prueba. Los intervalos de confianza son amplios y las diferencias de una o dos preguntas no son concluyentes. La revisión de Erika sigue pendiente y algunas correcciones las aplicó un solo revisor.
5. **Ruido del juez.** RAGAS depende de un modelo local que falla en algunas muestras y cambia con el timeout y la concurrencia. La comparación con la arquitectura anterior cambia juez y embedding, así que no es completamente equivalente.
6. **Hardware.** La estación tiene 16 GB de VRAM y 31 GB de RAM. El contexto de 116K ocupa 13,9 GB de VRAM y deja poco margen; la evaluación y la generación compiten por memoria y el sistema operativo puede matar procesos bajo presión.
7. **Alcance.** El sistema está diseñado para un corpus concreto de prensa y redes. No es un RAG de propósito general y las decisiones de limpieza, chunking y compuerta no se transfieren sin medir.

XI. Conclusiones y trabajo futuro

El sistema final responde 37 de 38 preguntas respondibles, se abstiene en las 5 sin respuesta y acierta 33 de 37 en revisión manual. En conversación alcanza 14 de 14 con compuerta de dominio y 11 de 14 sin ella. RAGAS reporta 0,952 de fidelidad, 0,881 de precisión del contexto y 0,921 de cobertura. La latencia media es de 6,8 s sin razonamiento y 6,5 s con razonamiento, con un modelo local de 7,5B y un reranker en CPU.

La primera lección es metodológica: el piloto sobreestima la recuperación. Con el mismo codificador y las mismas preguntas, Recall@10 cae de 0,947 a 0,684 al pasar de 1.932 a 161.868 documentos. Cualquier comparación de codificadores o de chunking hecha solo sobre una muestra pequeña habría elegido la configuración equivocada. La segunda es que el reordenamiento rinde mejor sobre un conjunto corto: reordenar el top-5 sube el nDCG 5,6 puntos y el P@1 13,1 sin perder cobertura, mientras que reordenar 20 o 50 candidatos pierde recall y multiplica la latencia. La tercera es que la limpieza conservadora basta: ninguna variante supera a otra de forma significativa, y la ganancia real es eliminar páginas de error, no reescribir el texto.

La cuarta lección es que la abstención barata es la compuerta. Rechazar antes de generar cuesta decenas de milisegundos y evita que el modelo improvise sobre temas ajenos. La quinta es que la evaluación automática necesita cobertura: una media de RAGAS con 4 valores de 38 invirtió una conclusión sobre la precisión del contexto. Reportar n junto a cada media no es un detalle de estilo; cambia lo que se concluye.

El trabajo futuro tiene cuatro frentes. Primero, cerrar los dos fallos de recuperación: g-046 exige revisar la consulta o el índice, y g-031 requiere un conjunto de candidatos mayor con un coste que hoy no se justifica. Segundo, mejorar la corrección de la respuesta, la métrica más débil de RAGAS. Tercero, completar la revisión del gold con la cuarta anotadora y ampliar el conjunto de prueba para reducir los intervalos de confianza. Cuarto, probar un chunking distinto y una reindexación completa, que es la única vía que queda para atacar la pérdida de contexto sin sacrificar cobertura.

XII. Reproducibilidad

XII-A. Hardware y software

El sistema se ejecutó en una estación con Ryzen 9 5950X, 31 GB de RAM y una GPU AMD Radeon RX 9070 XT de 16 GB con ROCm 7.2.4. Los contenedores usan Python 3.12, Milvus 2.6.6, PyTorch 2.10 con ROCm, FastAPI, Streamlit y Ollama. RAGAS 0.4.3 y LangChain 0.3 corren en un contenedor aparte con Python 3.12 y `BAAI/bge-m3` en CPU.

XII-B. Comandos

El stack se levanta con un comando:

```
docker compose up -d --build
docker compose ps
curl http://localhost:8010/health
```

El servicio `prewarm` carga el generador, el embedding y el reranker cuando la API está sana. La evaluación completa se ejecuta con:

```
python src/scripts/run_evaluation.py
```

Ese pipeline corre la evaluación de desempeño en el worker GPU, la de conversación contra la API y RAGAS en el contenedor de evaluación. Los resultados se escriben en `reports/tablas/` y la comparación en `reports/tablas/evaluacion_comparativa.md`.

XII-C. Artefactos

El gold consolidado está en `eval/gold/gold_standard.jsonl`, con su versión ampliada en `gold_standard_ampliado.jsonl`. Las tablas de recuperación, respuesta, conversación, compuerta y RAGAS están en `reports/tablas/`. Las figuras del artículo están en `reports/figuras/`. Los registros crudos de cada corrida están en `reports/corridas/` y los modelos de la compuerta en `modelos/`.

XII-D. Datos

El corpus completo no se publica por derechos de las fuentes. Se incluyen el gold, las anotaciones, la muestra del piloto y los Parquet de evaluación necesarios para reproducir las métricas. Las cifras de este artículo se pueden verificar contra esos artefactos sin volver a descargar el corpus.

Referencias

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *NeurIPS Datasets and Benchmarks Track*, 2021, arXiv:2104.08663.

[3] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023, arXiv:2210.07316.

[4] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin, "MIRACL: A multilingual retrieval dataset covering 18 diverse languages," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1114–1131, 2023, arXiv:2210.09984.

[5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[6] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, arXiv:2311.09476.

[7] A. Salemi and H. Zamani, "Evaluating retrieval quality in retrieval-augmented generation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, arXiv:2404.13781.

[8] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, P. Li *et al.*, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2408.08067.

[9] Zilliz, "Milvus documentation," https://milvus.io/docs, 2026.

[10] Ollama, "Ollama: Get up and running with large language models locally," https://github.com/ollama/ollama, 2026.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[12] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[13] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.

[14] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," 2024.

[15] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.

[16] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[17] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, arXiv:2307.03172.

[18] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, and T. Zhang, "Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," *arXiv preprint arXiv:2401.00396*, 2024.

[19] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com, 2026.

[20] Streamlit Inc., "Streamlit documentation," https://docs.streamlit.io, 2026.